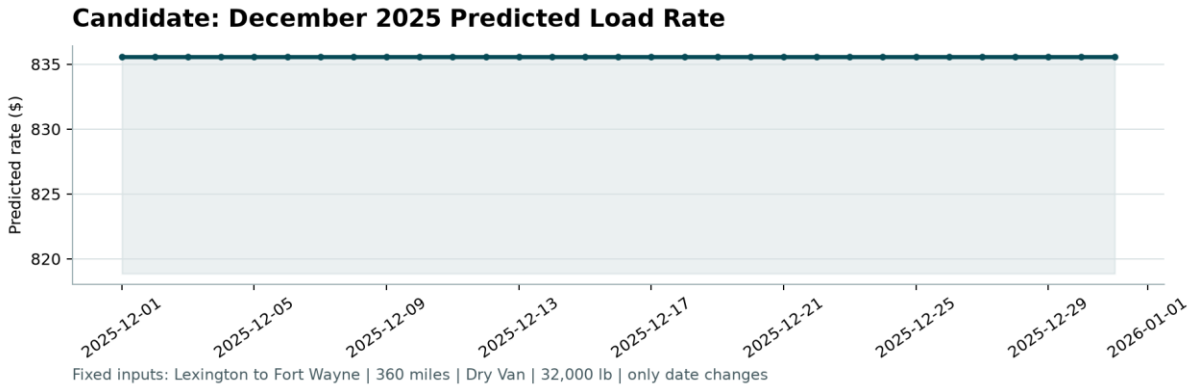
Freight Rate Prediction Report

By Leatile King Baaitse

My model predicts freight rates with an average error of about *5.5%* on data it had never seen. The December chart (fixed lane, Lexington → Fort Wayne) comes out at *$835.58 every day*, and Section 5 explains why that flat line is as accurate and answer as it could be.



[December 2025 predicted load rate · Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb](scorer_results/candidate_december.png)

1. How I split the data and checked my work:
    - The setup has a built-in time trap: the labelled data covers January–October 2025, but the 12,000 loads I must predict are all November–December 2025. So I validated the way the real world will judge the model; that is train on the past, test on the future:
        - Training side: January–August 2025 (~38,000 loads).
        - Test side: September–October 2025 (~10,000 loads), never touched during training.
        - Score: MAPE — the average percentage by which predictions miss. 5.46% means *"off by about five and a half percent on a typical load"*, which is fair to use when loads range from $57 to $25,000.

    I deliberately did not use random cross-validation (shuffling the data into random folds). Shuffling would let October loads leak into training, making the score look better than the model really is. September–October is also the closest available stand-in for the November–December period I actually have to predict. Every data-cleaning statistic was computed on the training side only, then applied unchanged to the test side.

2. What the data showed:
    - Price is mostly distance times a per-mile rate. Distance alone explains the bulk of every rate (correlation 0.91). The per-mile rate falls as trips get longer

(about $2.92/mile under 150 miles, down to $1.90/mile past 2,500 miles) and depends on the trailer: refrigerated trailers cost most (~$2.31/mile), then flatbeds (~$2.22), then dry vans (~$2.05).

- Weight barely matters. Heavier loads cost only slightly more; roughly an 8% difference from the lightest to the heaviest loads on otherwise identical trips. Useful as a small correction, not a driver.

- `quote_signal` helps a little. It's a weak/timid per-load signal, not a leak of the answer. Kept, but small positive effect.

- There is a weekly market rhythm. Average market levels swing from ~0.85 on Sundays to ~1.18 mid-week, and average rates follow that wave. This looked promising, and section 4 describes how I tested it and what happened.

3. Data problems I found and fixed:

DATA ISSUE	FREQUENCY / SCOPE	RESOLUTION STRATEGY
Weight missing	300 training / 165 validation rows	Filled with the typical weight for that trailer type.
Weight negative (e.g., -36,559 lb)	~300 training / ~145 validation rows	Treated as sign typos: the sizes match the positive weights almost exactly (averages 31,724 vs 31,415 lbs), so the minus sign was dropped.
Weight capped at exactly 47,500	~2.5% of rows	Left alone — that is the legal maximum, a real value.
Market index missing	~1% of rows	Filled with that day's typical value.
Extreme rates (up to $25,000+)	~0.6% of rows	Predicted on a logarithmic scale so outliers cannot hijack training; no rows deleted.
New cities and routes in validation	8 unseen cities; 35% of validation routes never appear in training	Used smoothed historical averages per city/route that gracefully fall back to global averages for unseen ones — instead of brittle one-hot codes.

4. The model and why this one:

    - A LightGBM gradient-boosting model with 8 inputs: distance, its inverse (capturing the falling per-mile rate), trailer type, smoothed historical rate levels for the lane and cities, cleaned weight and the quote signal.

    - Trained on log-rates with a percentage-error objective, 110 rounds (chosen on the holdout, then refit on all 48,000 rows for the final submission).

    Two decisions mattered most:

    1. Objective and target. Optimizing percentage error on log-rates scored 5.49%; plain squared-error scored 5.54%; predicting raw dollars directly was far *worse at 6.26%*.

    2. Killing the market/calendar features. I built the weekly-market feature set from Section 2 and tested it head-to-head on the holdout (`src/ablate_market.py`): without it 5.60%, with it 6.36%; adding market and calendar inputs made things 0.76 points worse. The weekly wave is real in aggregate but too noisy per load; the model chased noise. Ergo; calendar features were dropped entirely.

    3. Small, simple trees won (larger ones just memorized noise), and results were identical across random seeds, so no ensemble was needed.

5. The December chart · why it's flat:

    The December file holds everything constant; that is same cities, same 360 miles, same trailer, same weight; and varies only the date. But Section 4 proved date-based inputs *hurt* accuracy, so the model honestly has no date effect to draw: every December day predicts *$835.58*. That sits squarely inside this exact lane's training history ($823 ± $46 across 21 loads). A chart with lively day-to-day wiggles would be inventing precision the validation proved I don't have. The flat line is the finding.

6. How to reproduce everything:
    ``
    $python\ -m\ pip\ install\ -r\ requirements.txt$
    $python\ src/model.py$
    $python\ score.py\ --predictions\ validation_predictions.csv\ \backslash$
    $--december-predictions\ data/december_chart_inputs.csv$
    ``
    This regenerates `*validation_predictions.csv*`, refills the December file, prints the 5.46% backtest score, and redraws the chart above.